

IES JULIO VERNE

BARGAS (TOLEDO)

Ciclo Formativo de Grado Superior

DESARROLLO DE APLICACIONES WEB (DAW)

MEMORIA DE PRÁCTICAS

Práctica 2. Dependencias

Alumno: Dayra Daniela Gallegos Velasquez

Ariadna Lilibeth Montero Matínez

Adrián Toledano López-Gálvez

Asignatura: Desarrollo de Aplicaciones en Entorno Servidor

Profesor/a: Mario Marugán

Curso académico: 2026 – 2027

Fecha: 20/10/2026

CONTENIDO

1. OBJETIVOS 2
2. DESARROLLO DE LA PRÁCTICA 2
3. RESULTADOS OBTENIDOS 3
4. DIFICULTADES ENCONTRADAS Y SOLUCIONES APLICADAS 3
5. CONCLUSIONES 3

Enlace GitHub: https://github.com/ariadnamonterom-sudo/P5-G1.git

1. OBJETIVOS

- Usar el patrón **MVC** para separar la lógica de negocio, presentación y datos.
- Diseño y la manipulación de objetos
- Detección y modelado de relaciones entre clases
- Uso de anotaciones de Spring MVC.
- Inyección de dependencias.
- Aplicación de todos los conocimientos sobre programación aprendidos hasta el momento.
- Uso de modificadores de visibilidad
- Uso de excepciones.
- Uso de patrones de desarrollo.

2. DESARROLLO DE LA PRÁCTICA

En esta práctica se van a ver cómo se desarrollan los diferentes controladores y endpoints para una aplicación de gestión de una clínica con pacientes, sus partes médicos y tratamientos.

Para ello se han utilizado los conocimientos aprendidos en el Tema 2 de Desarrollo Web en Entorno Servidor, concretamente el tema orientado a inyección de dependencias.

1. Diseño

El diagrama de clases del proyecto ha sido diseñado en Visual Paradigm siguiendo una arquitectura por capas en Spring Boot, lo que facilita la separación de responsabilidades, la modularidad y el desacoplamiento mediante interfaces e inyección de dependencias.

Conexión entre capas (controllers → services → repositories):
Cada controlador se conecta con la interfaz de su servicio correspondiente (PatientService, MedicalRecordService, TreatmentService), sin depender de la clase directa. A su vez, las clases que implementan los servicios se comunican con las interfaces de los repositorios.

models:
Las clases Patient, MedicalRecord y Treatment están conectadas con todas las capas anteriores, los servicios procesan y los repositorios guardan o consultan. Además, entre ellas guardan relación: MedicalRecord se vincula a Patient, y Treatment se vincula a MedicalRecord.

config y util:
El paquete config se encarga de instanciar y enlazar los repositorios concretos para que puedan inyectarse en los servicios, mientras que util aporta clases auxiliares (RequestInfo, UserSession) que dan soporte al flujo general de la aplicación.

com.desarrolloweb.entornoservidor.tema2practica1
- model
- repository
- service
- controller
- config

2. Código fuente (Enunciado de la práctica)

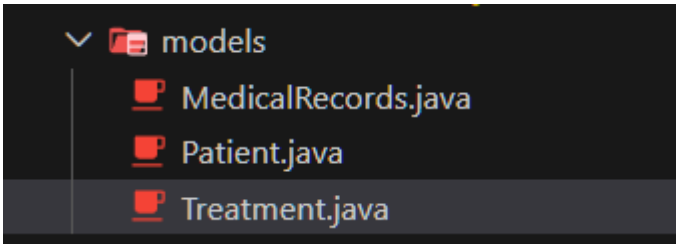
La práctica consta de una aplicación que va a gestionar una clínica médica. Para ello hemos utilizado 'Spring' para darle forma al modelo-vista-controlador (MVC).

El enunciado proporcionado es:

ClinicaMed es una aplicación web (backend) para gestionar pacientes y su atención médica en una clínica. Su objetivo principal es organizar la información de pacientes, registrar sus visitas (partes médicos) y los tratamientos que se les administran, permitiendo consultar, agregar y relacionar todos estos datos de manera estructurada.

Esta es la estructura que se ha facilitado para poder elaborar la aplicación:

Bloque 1 – Modelos



En cada modelo se ha implementado la interfaz Cloneable para poder clonar objetos de esos modelos y además se han sobreescrito los métodos toString() y clone().

```
// CLASE MEDICALRECORD//
public class MedicalRecords implements Cloneable{
```

Este es un ejemplo de la clase MedicalRecords donde se implementa la interfaz Cloneable.

Bloque 2 – Repositories



Estos son los repositorios que se han utilizado. Los tres primeros son interfaces que luego se han implementado en cada una de los repositorios con el mismo nombre.

```
package com.desarrolloweb.entornoservidor.tema2practica1.repositories;

import java.util.List;

import com.desarrolloweb.entornoservidor.tema2practica1.models.Patient;
// Creamos la interfaz
public interface IPatientRepository {
    // Declaramos los métodos que va a manejar la interfaz de los repositorios
    public Patient findById(Long id);

    public List<Patient> findAll();

    public int save(Patient paciente);
}
```

Ejemplo de una de las interfaces de repositorios.

Practica > tema2practica1 > src > main > java > com > desarrolloweb > entornoservidor > tema2practica1 > repositories > PatientRepository.java > Language

```java
package com.desarrolloweb.entornoservidor.tema2practica1.repositories;

import java.util.ArrayList;
import java.util.List;
import java.util.concurrent.atomic.AtomicLong;

import org.springframework.context.annotation.Primary;
import org.springframework.stereotype.Repository;
import com.desarrolloweb.entornoservidor.tema2practica1.models.Patient;

@Repository // indicamos que es un repositorio
@Primary // Indicamos con la etiqueta de Primary que este repositorio es el primero que tiene que utilizar
public class PatientRepository implements IPatientRepository{
    // Declaramos la lista de pacientes y el generador automatico de id
    private List<Patient> pacientes = new ArrayList<>();
    private AtomicLong idGenerator = new AtomicLong();

    // Sobreescribimos el método de la interfaz
    @Override
    public Patient findById(Long id) {
        return pacientes.stream().filter(m -> m.getId().equals(id))
                .findFirst().orElse(null);
    }

    // Sobreescribimos el método de la interfaz
    @Override
    public List<Patient> findAll() {
        return pacientes;
    }

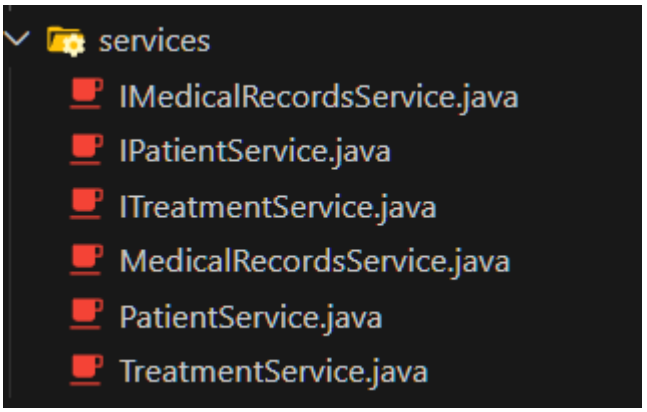
    // Sobreescribimos el método de la interfaz
    @Override
    public int save (Patient patient) {
        patient.setId(idGenerator.incrementAndGet());
        pacientes.add(patient);
        return pacientes.size();
    }
}

```

Ejemplo de un Repositorio donde se puede ver que implementa su interfaz correspondiente y los métodos que sobreescribe.

Bloque 3 – Servicios

7. **Crear interfaces: IPatientService, IMedicalRecordService, ITreatmentService.**



Distribución de los Servicios en la aplicación. Se pueden ver Interfaces en las tres primeras clases que luego se implementan en las clases Servicio de más abajo.

```java
package com.desarrolloweb.entornoservidor.tema2practica1.services;

import java.util.List;

import org.springframework.beans.factory.annotation.Qualifier;
import org.springframework.stereotype.Service;

import com.desarrolloweb.entornoservidor.tema2practica1.models.Patient;
import com.desarrolloweb.entornoservidor.tema2practica1.repositories.IPatientRepository;

// Indicamos que esto es un servicio
@Service
public class PatientService implements IPatientService{ // Implemeentamos la interfaz
    //@Autowired -- Si lo ponemos NO FUNCIONA
    private IPatientRepository patientRepository;

    // Declaramos el constructuctor con la anotación Qualifier
    public PatientService(@Qualifier("patientJson")IPatientRepository patientRepository) {
        this.patientRepository = patientRepository;
    }

    // Sobreescribimos el método de la interfaz
    @Override
    public Patient getById(Long id) {
        return patientRepository.findById(id);
    }

    // Sobreescribimos el método de la interfaz
    @Override
    public List<Patient> getAll() {
        return patientRepository.findAll();
    }

    // Sobreescribimos el método de la interfaz
    @Override
    public int add(Patient paciente) {
        return patientRepository.save(paciente);
    }
}

```

Ejemplo de una de las clases Servicio donde se puede ver que implementa la interfaz correspondiente y los métodos que sobreescribe.

8. **Implementar servicios: PatientService, MedicalRecordService, TreatmentService.**

Practica > tema2practica1 > src > main > java > com > desarrolloweb > entornoservidor > tema2practica1 > services > IMedicalRecordsService.j

```java
package com.desarrolloweb.entornoservidor.tema2practica1.services;

import java.util.List;

import com.desarrolloweb.entornoservidor.tema2practica1.models.MedicalRecords;

// Creamos la interfaz
public interface IMedicalRecordsService {
    // Declaramos los métodos que va a manejar la interfaz de los servicios
    public MedicalRecords getId(Long id);

    public List<MedicalRecords> getAll();

    public int add(MedicalRecords medicalRecord);

    public List<MedicalRecords> getRecordsByPatientId(Long patientId);
}

```

Ejemplo de una Interfaz de Servicios que luego se ha implementado en su servicio correspondiente.

9. Inyección avanzada:

```java
@Repository // indicamos que es un repositorio
@Primary // Indicamos con la etiqueta de Primary que este repositorio es el primero que tiene que utilizar
public class PatientRepository implements IPatientRepository{
    // Declaramos la lista de pacientes y el generador automatico de id
    private List<Patient> pacientes = new ArrayList<>();
    private AtomicLong idGenerator = new AtomicLong();
```

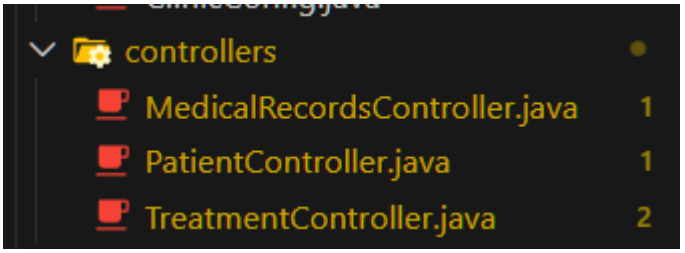
Utilización de @Primary en el repositorio PatientRepository

```java
    // Declaramos el constructuctor con la anotación Qualifier
    public PatientService(@Qualifier("patientJson")IPatientRepository patientRepository) {
        this.patientRepository = patientRepository;
    }
```

Utilización de @Qualifier en el servicio PatientService

Bloque 4 – Controladores

10. Crear controladores REST:



Se han creado los 3 controladores correspondientes.

11. Métodos comunes:

```java
@RestController
@RequestMapping ("/api/medicalRecords") // Definimos la ruta que tiene que mapear
public class MedicalRecordsController {

    // Inyectamos la dependencia del sevicio
    @Autowired     Convert @Autowired field into Constructor Parameter
    public IMedicalRecordsService medicalRecordsService;

    // Hacemos un GetMapping para buscar por id
    @GetMapping("/{id}")
    public MedicalRecords getMedicalRecords(@PathVariable Long id) {
        MedicalRecords medicalRecords = medicalRecordsService.getId(id);
        return medicalRecords;
    }

    // Hacemos un GetMapping para listar todo
    @GetMapping
    public List<MedicalRecords> getListMedicalRecords() {
        return medicalRecordsService.getAll();
    }

    // Hacemos un PostMapping para poder añadir más objetos
    @PostMapping
    public String save(@RequestBody MedicalRecords medicalRecords) {
        int id = medicalRecordsService.add(medicalRecords);
        return "redirect:/medicalRecords/" + id;
    }
```

Ejemplo de un RestController, en concreto del MedicalRecordsController donde se pueden ver los 3 métodos que se han utilizado. Los otros 2 controladores (PatientController y TreatmentController) siguen la misma estructura de endpoints.

12. **Inyectar servicios mediante constructor.**

```java
// Inyectamos la dependencia del sevicio por constructor
public final IMedicalRecordsService medicalRecordsService;

MedicalRecordsController(IMedicalRecordsService medicalRecordsService) {
    this.medicalRecordsService = medicalRecordsService;
}
```

Ejemplo de la inyección de un servicio mediante constructor.

Bloque 5 – Relaciones entre entidades

13. **Consultas basadas en relaciones:**

```java
// Sobreescribimos el método de la interfaz findById()
public List<MedicalRecords> findByPatientId(Long patientId) {
    return medicalRecords.stream().filter(t -> t.getPatientId() != null && t.getPatientId().equals(patientId)).collect(Collectors.toList()
}
```

```java
// Sobreescribimos el método de la interfaz
public List<Treatment> findByMedicalRecords(Long medicalRecordId) {
    return treatmentList.stream().filter(t -> t.getMedicalRecordId() != null && t.getMedicalRecordId().equals(medicalRecordId))
    .collect(Collectors.toList());
}
```

Bloque 6 – Configuración externa

14. **Crear application.properties:**
clinic.name=Clinica Med
maxAppointmentsPerDay=20

```
Practica > tema2practica1 > src > main > resources > application.properties
1   spring.application.name=tema2practica1
2   clinic.name=Clinica Med     'clinic.name' is an unknown property.
3   maxAppointmentsPerDay=20    'maxAppointmentsPerDay' is an unknown property.
4
5
```

15. **Crear AppConfig con @Component y @Value para leer las propiedades.**

Practica > tema2practica1 > src > main > java > com > desarrolloweb > entornoservidor > tema2practica1 > config > AppConfig.java >

```java
package com.desarrolloweb.entornoservidor.tema2practica1.config;
import org.springframework.beans.factory.annotation.Value;
import org.springframework.context.annotation.PropertySource;
import org.springframework.stereotype.Component;

// Indicamos que es component
@Component
@PropertySource("classpath:application.properties")
public class AppConfig {

    // Indicamos un Value para el nombre de la app
    @Value("${spring.application.name:tema2practica1}")
    private String appName;

    // Indicamos un Value para el valor por defecto
    @Value("${clinic.name:Clinica Med}")
    private String appNameDefault;

    // Indicamos un valor para el máximo por dia
    @Value("${maxAppointmentsPerDay:20}")
    private int maxAppointmentsPerDay;

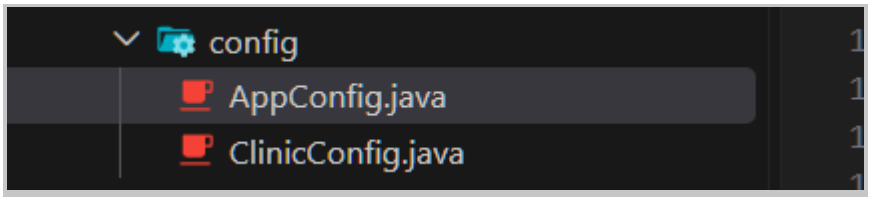
    // Les hacemos los getters para acceder a ellos
    public String getAppName() {
        return appName;
    }

    public int getMaxAppointmentsPerDay() {
        return maxAppointmentsPerDay;
    }

    // Sobreescribimos el método toString
    @Override
    public String toString() {
        return "AppConfig {" +
                "springAppName='" + appName + '\'' +
                ", clinicName='" + appNameDefault + '\'' +
                ", maxAppointmentsPerDay=" + maxAppointmentsPerDay +
                '}';
    }
}
```

Bloque 7 – Configuración de Beans

18. **Crear ClinicConfig con @Configuration.**



19. **Registrar beans de PatientRepository, MedicalRecordRepository y TreatmentRepository con @Bean.**

```java
package com.desarrolloweb.entornoservidor.tema2practica1.config;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import com.desarrolloweb.entornoservidor.tema2practica1.repositories.MedicalRecordsRepository;
import com.desarrolloweb.entornoservidor.tema2practica1.repositories.PatientRepositoryJSON;
import com.desarrolloweb.entornoservidor.tema2practica1.repositories.TreatmentRepository;

// Indicamos que es Configuration
@Configuration
public class ClinicConfig {

    // Creamos un Bean para cada Repositorio. Cada método devuelve una instancia nueva de cada uno
    @Bean("patientJSON")
    public PatientRepositoryJSON patientRepositoryJSON(AppConfig appConfig) {
        return new PatientRepositoryJSON();
    }

    @Bean
    public MedicalRecordsRepository medicalRecordsRepository() {
        return new MedicalRecordsRepository();
    }

    @Bean
    public TreatmentRepository treatmentRepository() {
        return new TreatmentRepository();
    }


}
```

Bloque 8 – Scopes (opcional)

20. **@RequestScope → componente para datos temporales por petición. Endpoint informativo que muestra la configuración y un campo temporal de la petición.**

```java
package com.desarrolloweb.entornoservidor.tema2practica1.util;

import java.time.LocalDateTime;

import org.springframework.stereotype.Component;
import org.springframework.web.context.annotation.RequestScope;

// Indicamos que es un componente y un RequestScope
@Component
@RequestScope
public class RequestInfo {

    // Declaramos las propiedades de fecha y descripcion
    private final LocalDateTime requesTime;
    private String description;

    // Inicializamos en el constructor los parámetros
    public RequestInfo() {
        this.requesTime = LocalDateTime.now();
        this.description = "Petición procesada correctamente";
    }

    // Solo hacemos el getter del LocalDateTime ya que siempre va a acceder con la función now()
    public LocalDateTime getRequestTime() {
        return requesTime;
    }

    // Setters y Getters de la descripción
    public String getDescription() {
        return description;
    }

    public void setDescription(String description) {
        this.description = description;
    }
}

```

21. **@SessionScope → componente para mantener datos del paciente durante la sesión.**

```java
package com.desarrolloweb.entornoservidor.tema2practica1.util;

import org.springframework.stereotype.Component;
import org.springframework.web.context.annotation.SessionScope;

import com.desarrolloweb.entornoservidor.tema2practica1.models.Patient;

// Indicamos que esto es un componente y un SessionScope
@Component
@SessionScope
public class UserSession {

    // Declaramos la propiedad
    private Patient sessionPatientHolder;

    // Inicializamos la propuedad en el constructor
    public UserSession() {
        this.sessionPatientHolder = new Patient();
    }

    // Hacemos el getter y setter
    public Patient getSessionPatientHolder() {
        return sessionPatientHolder;
    }

    public void setSessionPatientHolder(Patient sessionPatientHolder) {
        this.sessionPatientHolder = sessionPatientHolder;
    }

    // Sobreescribimos el método toString()
    @Override
    public String toString() {
        return sessionPatientHolder.toString();
    }


}

```

Tabla 1. Tecnologías usadas.

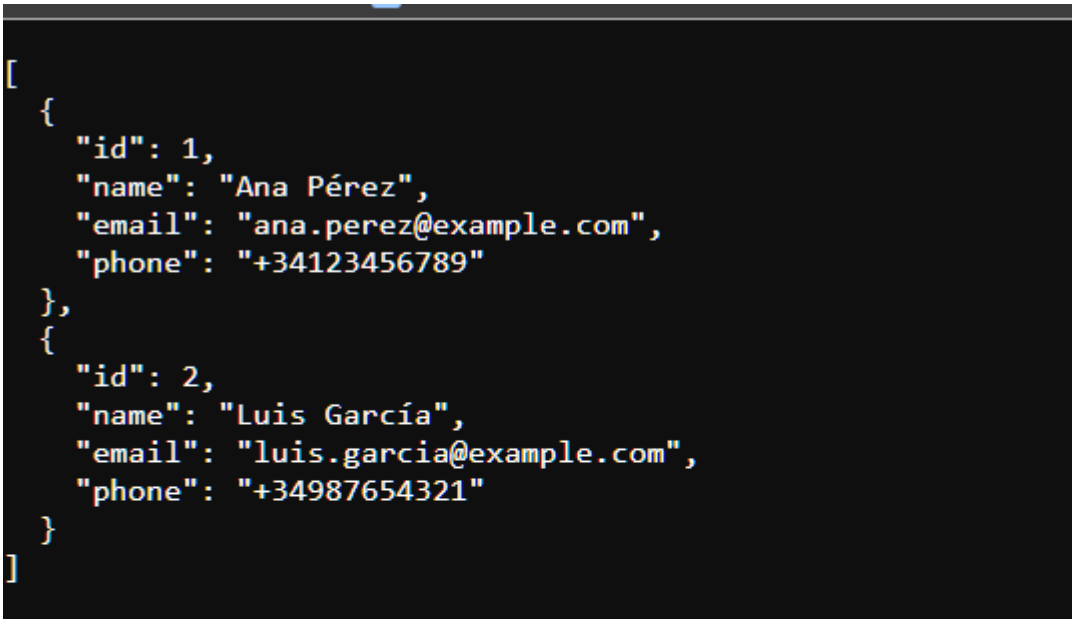
Tecnologías y herramientas empleadas	
Tecnología	**Descripción**
Java [1]	Desarrollo de la lógica de negocio
GitHub [4]	Repositorio del proyecto

3. RESULTADOS OBTENIDOS

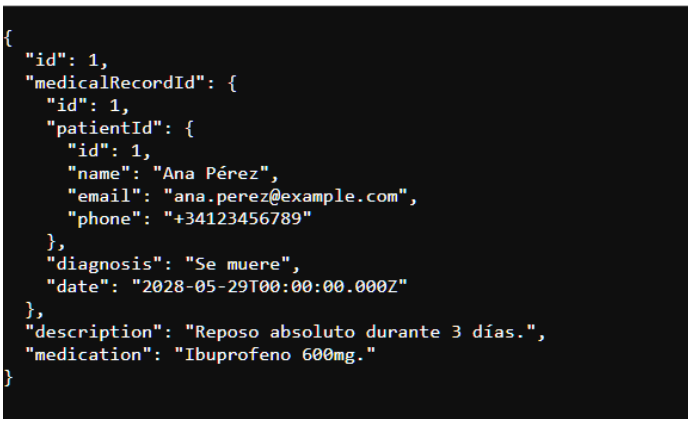
En este apartado se muestran las evidencias que demuestran el correcto funcionamiento de la aplicación o solución desarrollada.

Además, se presentan los resultados alcanzados, las pruebas realizadas y las comprobaciones efectuadas para verificar que se cumplen los requisitos establecidos.

Evidencias de ProductRESTController (Modelo Controlador)



Evidencia en formato JSON de la lista de pacientes con el MedicalRecord y el Treatment añadido.



4. DIFICULTADES ENCONTRADAS Y SOLUCIONES APLICADAS

Uno de los principales problemas ha sido la configuración y el registro de los beans en las clases de configuración, así como entender cómo Spring debe gestionarlos y cargarlos correctamente en el contenedor de dependencias.

Para solucionar esto, se ha recurrido a los apuntes y a la documentación oficial, revisando la correcta utilización de las anotaciones de configuración para evitar fallos al arrancar la aplicación o al procesar los datos de la clínica.

5. CONCLUSIONES

Como conclusión final, Spring hace mucho más sencillo el control del patrón Modelo-Vista-Controlador (MVC) y la inyección de dependencias, lo que hace posible que la aplicación sea escalada y mantenida con mayor facilidad. Además, con la estructura bien definida en la gestión de pacientes, partes médicos y tratamientos, hay más facilidad a la hora de localizar y corregir cualquier error de lógica o programación.